

Лабораторна робота 01

Надійний typed LLM-клієнт

Курс «Прикладні ШІ-агенти та системи на основі великих мовних моделей» · 2026/2027

Лекції	1–5
Аудиторний час	3 год; самостійна робота 6 год
Формат	індивідуально
Вартість	0 грн: локальна модель або stub, платні API не потрібні

Мета

Побудувати клієнт до мовної моделі, який перетворює недовірений текст на валідований типізований об'єкт або на контрольовану відмову, і жодного разу не повертає неперевірений результат як успішний.

Теоретичні відомості

Мовна модель повертає розподіл над продовженнями, а не значення функції. Та сама пара «інструкція + запит» може дати різні виходи, і причина не лише в семплінгу: на результат впливають версія моделі, склад і порядок контексту, а на GPU ще й пакетування запитів разом із неасоціативністю операцій з рухомою комою [5, 6]. Тому текст відповіді не є інтерфейсом, а межа між моделлю і кодом описується контрактом.

Гарантій буває три рівні, і вони не взаємозамінні. Інструкція в промпті «відповідай у JSON» не гарантує нічого: вона змінює ймовірності, але не забороняє невалідний вихід. Валідація після генерації гарантує, що некоректний результат не пройде далі, ціною повторних викликів. Обмежене декодування (constrained decoding) будує граматику допустимих продовжень і на кожному кроці забороняє токени, що виводять за межі схеми; синтаксична валідність тоді гарантована самою процедурою генерації [3, 4]. Провайдерські API реалізують це під назвою strict mode, локально ту саму роль виконують GBNF-граматики в llama.cpp або бібліотека Outlines.

Схема описує форму, а не зміст. Модель Pydantic породжує JSON Schema, у якій важливі три речі: перелік required, типи полів і заборона зайвих полів (additionalProperties: false). Остання не косметична: без неї модель домальовує поля, яких немає в контракті, і вони мовчки проходять далі. Але навіть строга схема гарантує лише те, що поле date існує і є рядком, а не те, що дата правильна. Доменні правила перевіряє код, і ніхто інший за нього цього не зробить [1, 2].

Помилки, що виникають на цій межі, належать до чотирьох різних класів, і плутанина між ними — типова причина того, що сервіс або зациклюється, або мовчки псує дані.

Клас	Приклад	Реакція
Транспорт	timeout, розрив з'єднання	повтор із відкатом
Схема	відсутнє поле, не той тип	одна спроба raise
Доменна	дата в минулому, від'ємна сума	контрольована відмова
Відмова моделі	«не можу відповісти»	контрольована відмова

Повтор безпечний для читання і небезпечний для запису. Якщо виклик змінює стан, потрібен ключ ідемпотентності, обчислений до запиту й перевірений у власному сховищі, інакше

повтор після таймауту створює дублікат. Затримка між спробами зростає експоненційно і обов'язково має випадкову складову (jitter): без неї всі клієнти повторюють синхронно і роблять перевантаження гіршим. Понад це в клієнті потрібні загальний бюджет часу на запит, запобіжник (circuit breaker), який припиняє звертання до сервісу, що стабільно падає, і резервна модель на випадок недоступності основної [5].

Нарешті, кожен запит вимірюється: кількість вхідних і вихідних токенів, час до першого токена і повний час. Без цього обліку вартість зростає непомітно, а порівняння двох варіантів промпта не має підстав.

Хід роботи

Позначка *ауд.* — крок виконується на занятті, *сам.* — самостійно в межах СРС. Аудиторна частина розрахована рівно на 3 години, решта кроків — домашні.

1. **Середовище (30 хв, ауд.).** Підняти локальну модель і окремо режим `stub`, що відтворює записані відповіді з файлу. *Перевірка перед наступним кроком:* усі тести проходять у режимі `stub` без мережі. Якщо для тестів потрібна жива модель — контракт спроектовано неправильно.
2. **Контракт (40 хв, ауд.).** Описати `Pydantic`-модель, вивести з неї `JSON Schema`. *Перевірка:* у схемі є `required` з усіма полями і `additionalProperties: false`. Вручну підсунути об'єкт із зайвим полем — має впасти валідація, а не пройти далі.
3. **Розбір з однією спробою відновлення (40 хв, ауд.).** Невалідний `JSON` → повторний запит із текстом помилки валідатора. *Перевірка:* у логах видно, що друга спроба отримала саме текст помилки, а не той самий промпт.
4. **Чотири класи помилок (30 хв, ауд.).** Транспорт, схема, доменна, відмова моделі. *Перевірка:* кожен клас відтворюється окремим випадком із набору, і для кожного видно рішення про повтор.
5. **Надійність (40 хв, ауд.).** Таймаут на виклик, бюджет часу на запит, відкат із джигером. *Перевірка:* штучно сповільнити `stub` і показати в логах, що обробку зупинив саме бюджет, а не виняток.
6. **Ідемпотентність (30 хв, сам.).** Ключ операції рахується до виклику. *Перевірка:* два однакові запити поспіль дають один запис у сховищі.
7. **Набір і тести (60 хв, сам.).** 30+ випадків, 12+ тестів `pytest`. *Перевірка:* у наборі є обірваний `JSON` і відмова моделі — саме вони найчастіше не покриті.
8. **Вимірювання і звіт (40 хв, сам.).** Прогнати набір, звести метрики в одну таблицю, написати висновок до 150 слів.

Типова помилка порядку: писати тести після коду. Набір випадків з кроку 7 корисніше скласти одразу після кроку 2 — тоді контракт перевіряється на реальних поламаних даних.

Завдання

1. Підготувати середовище: локальна модель через `Ollama` або `llama.cpp` (3–8B, 4-бітний квант) і режим `stub`, який відтворює записані відповіді з файлу. Усі автоматичні перевірки мають проходити в режимі `stub`.
2. Описати контракт результату як модель `Pydantic`: не менше п'яти полів, серед них хоча б одне числове з діапазоном, одне з перелічуваного типу і одне необов'язкове. Заборонити додаткові поля. Вивести з моделі `JSON Schema` і додати її до звіту.
3. Реалізувати розбір відповіді з рівно однією спробою відновлення: у разі невалідного `JSON` повторний запит містить текст помилки валідації. Друга невдала спроба завершується контрольованою відмовою.
4. Реалізувати класифікацію помилок за чотирма класами з таблиці. Для кожного класу повернути типізований об'єкт відмови з полями: клас, повідомлення, чи доречний повтор.
5. Додати повтор з експоненційним відкатом і джигером, таймаут на окремий виклик і бюджет часу на весь запит. Показати в логах, що бюджет справді зупиняє обробку.

6. Реалізувати ідемпотентність: обчислити ключ операції до виклику, зберігати результати у файлі або SQLite, повторний запит з тим самим ключем не має створювати другий запис.
7. Підготувати набір з не менш ніж 30 синтетичних випадків, серед яких: коректні, з відсутнім полем, з полем неправильного типу, із зайвим полем, з порушенням доменного правила, з обірваним JSON і з відмовою моделі. Написати не менше 12 тестів `pytest`.
8. Виміряти на цьому наборі: частку валідних за схемою відповідей, частку успішних відновлень, частку контрольованих відмов, середню кількість токенів на запит і затримку p50 та p95. Звести в одну таблицю.
9. Здати: код і README з однією командою запуску, набір випадків, файл результатів, таблицю метрик і висновок обсягом до 150 слів про те, який клас помилок виявився найчастішим і чому.

Контрольні запитання

1. Чому температура 0 не гарантує однакового виходу на однаковому вході?
2. Що саме гарантує обмежене декодування і чого воно не гарантує?
3. Навіщо в схемі `additionalProperties: false`, якщо зайві поля можна просто ігнорувати?
4. Чим доменна помилка відрізняється від помилки схеми і чому реакція на них різна?
5. У яких випадках повторний виклик робить гірше, ніж відмова?
6. Що станеться, якщо у відкату немає випадкової складової?
7. Як обчислити ключ ідемпотентності для запиту, що містить довільний текст користувача?
8. Чому одна спроба відновлення, а не три?
9. Які метрики покажуть, що промпт погіршився після зміни, а які цього не покажуть?
10. Чому потокова передача (streaming) насамперед змінює час до першого токена і сприйману затримку, а не обсяг генерації — і за яких умов вона все ж змінює повний час відповіді?

Література

1. Pydantic. JSON Schema i Strict Mode.
2. JSON Schema Validation, draft 2020-12.
3. Willard B., Louf R. Efficient Guided Generation for Large Language Models. arXiv:2307.09702, 2023.
4. llama.cpp. GBNF Grammars.
5. Xiao T., Zhu J. Foundations of Large Language Models. arXiv:2501.09223, 2025. Розділ 5.
6. Jurafsky D., Martin J. H. Speech and Language Processing, 3rd ed. draft, 2026. Розділ 7.
7. OpenAI. Structured Outputs.
8. Anthropic. API errors and retry behaviour.
9. Brooker M. Exponential Backoff And Jitter. AWS Architecture Blog, 2015.

Походження шаблону

Макет адаптовано з Laboratory report template for Basic Chemistry at Stockholm University, Viktor Bengtsson, LPPL 1.3c: article, Times-подібна гарнітура, широкі поля, fancyhdr, booktabs.